

Part 1

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("data.csv", header=None)
X, y = df.iloc[:, :2].values, df.iloc[:, 2].values

def train_and_plot(ax, lr):
    np.random.seed(42)
    w, b = np.random.uniform(-1, 1, 2), np.random.uniform(-1, 1)

    ax.scatter(*X[y==0].T, c="red", edgecolors="black", s=20)
    ax.scatter(*X[y==1].T, c="blue", edgecolors="black", s=20)

    x = np.array([X[:,0].min()-0.05, X[:,0].max()+0.05])
    ax.plot(x, -(w[0]*x+b)/w[1], "r-", lw=2)

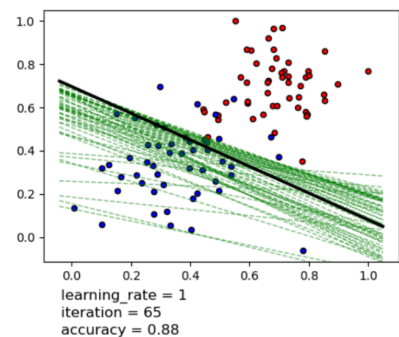
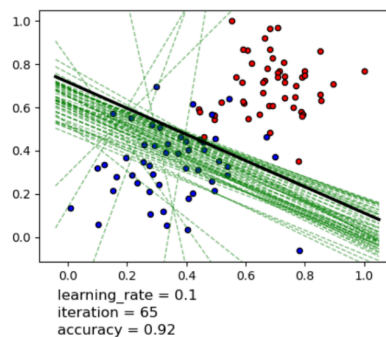
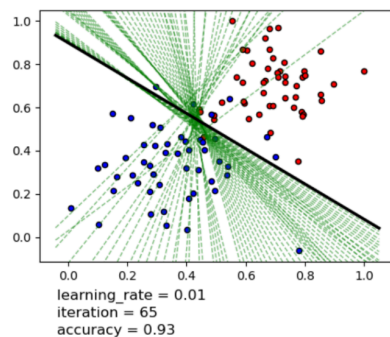
    for _ in range(65):
        for xi, yi in zip(X, y):
            p = 1 if np.dot(w, xi) + b >= 0 else 0
            if p != yi:
                if p == 0:
                    b += lr; w += lr * xi
                else:
                    b -= lr; w -= lr * xi
            ax.plot(x, -(w[0]*x+b)/w[1], "g--", lw=1, alpha=0.5)

    ax.plot(x, -(w[0]*x+b)/w[1], "k-", lw=2.5)
    acc = np.mean([(1 if np.dot(w, xi)+b >= 0 else 0) == yi for xi, yi in zip(X, y)])

    ax.set_ylim(X[:,1].min()-0.05, X[:,1].max()+0.05)
    ax.text(0.05, -0.15, f"learning_rate = {lr}", transform=ax.transAxes, fontsize=12)
    ax.text(0.05, -0.22, "iteration = 65", transform=ax.transAxes, fontsize=12)
    ax.text(0.05, -0.29, f"accuracy = {acc:.2f}", transform=ax.transAxes, fontsize=12)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for ax, lr in zip(axes, [0.01, 0.1, 1]):
    train_and_plot(ax, lr)

plt.subplots_adjust(bottom=0.3)
plt.show()
```



In Part 1, I implemented a perceptron using the heuristic update rule to classify the given dataset. The data consists of red and blue points, representing class 0 and class 1 respectively. From the scatter plot, the two classes appear to be mostly linearly separable, although some points are located close to the boundary, making perfect separation difficult.

I initialized the weights and bias randomly and trained the model for 65 iterations. For each data point, I made a prediction using a step function, and whenever a point was misclassified, I updated the weights and bias according to the given rules.

To observe how the model learns, I plotted the decision boundary at each iteration. The initial boundary was shown in red, the intermediate boundaries were shown as dashed green lines, and the final boundary was shown in black. From the graph, I could see that the boundary changed significantly in the early stages due to many misclassifications, but as the iterations progressed, the changes became smaller and the boundary gradually stabilized.

Looking at the final result, most of the blue points are located below the black decision boundary, while most of the red points are above it. This indicates that the model is able to separate the two classes reasonably well. However, a few points near the boundary are not perfectly classified.

I experimented with three different learning rates (0.01, 0.1, and 1), while keeping the number of iterations fixed at 65. With a learning rate of 0.01, the boundary moved smoothly and the training was stable, achieving an accuracy of about 0.93. With 0.1, the updates were slightly larger but still stable, resulting in an accuracy of about 0.92. However, with a learning rate of 1, the updates became too large, causing the boundary to change aggressively and reducing the accuracy to about 0.88.

Overall, I observed that smaller learning rates lead to more stable boundary updates and better performance, while larger learning rates make the model less stable and decrease accuracy.

Part 2

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("data.csv", header=None).values
X, y = df[:, :2], df[:, 2]
S = lambda z: 1 / (1 + np.exp(-z))

np.random.seed(42)
w, b = np.random.uniform(-1, 1, 2), np.random.uniform(-1, 1)
lr, epochs = 0.01, 100
x = np.array([X[:,0].min()-0.05, X[:,0].max()+0.05])
errs = []

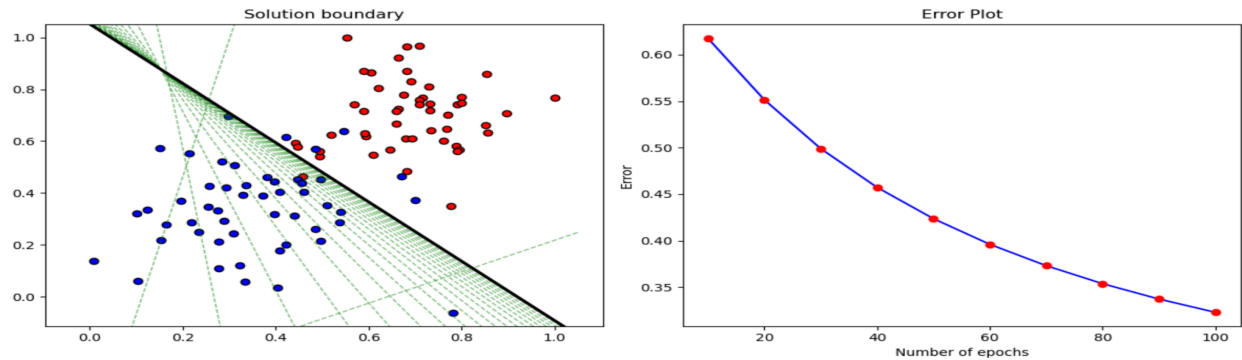
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
ax1.scatter(*X[y==0].T, c="red", ec="k")
ax1.scatter(*X[y==1].T, c="blue", ec="k")

draw = lambda c, ls, lw, a=1: ax1.plot(x, -(w[0]*x+b)/w[1], c=c, ls=ls, lw=lw, alpha=a)

draw("red", "-", 2)
for e in range(epochs):
    for xi, yi in zip(X, y):
        err = yi - S(w[0]*xi + b)
        w, b = w + lr*err*x, b + lr*err
    if e % 5 == 0: draw("green", "--", 1, 0.5)
    if (e+1) % 10 == 0:
        yhat = S(X[0]*w + b)
        errs.append(-np.mean(y*np.log(yhat+1e-9) + (1-y)*np.log(1-yhat+1e-9)))
draw("black", "-", 2.5)

ax1.set(title="Solution boundary", ylim=(X[:,1].min()-0.05, X[:,1].max()+0.05))
ep = range(10, epochs+1, 10)
ax2.plot(ep, errs, "b-")
ax2.scatter(ep, errs, c="red", zorder=5)
ax2.set(title="Error Plot", xlabel="Number of epochs", ylabel="Error")

plt.tight_layout()
plt.show()
```



In Part 2, I used gradient descent to train the perceptron instead of the heuristic update rule. One key difference is that the model uses a sigmoid function rather than a step function, so the output is no longer just 0 or 1, but a value between 0 and 1. This means the prediction can be interpreted like a probability, and every data point contributes to updating the weights and bias through the error term $(y - \hat{y})$.

During training, I observed how the decision boundary changed over time. At the beginning, the boundary moved quite significantly, but as the number of epochs increased, the changes became smaller and more stable. Compared to Part 1, the dashed green lines appeared smoother, which showed that the updates were more gradual. In the final result, most blue points were located below the boundary and most red points were above it, indicating that the model separated the two classes reasonably well.

Another important observation came from the error plot. By calculating the log loss every 10 epochs, I could clearly see that the error decreased as training progressed. The slope of the error curve was steep in the beginning and became flatter over time, which suggests that the model was gradually converging.

I also experimented with different learning rates. A smaller learning rate resulted in smoother and more stable convergence, while a larger learning rate caused the boundary to fluctuate more and made the error less stable. Overall, using gradient descent made the learning process easier to understand, since both the boundary movement and the error reduction could be observed directly.